Lab 2 Report

Summer 2026

Soeun Jang
Virginia Tech
sophiaj04@vt.edu

June 15, 2026

Contents

Introduction

In this lab, we study the sensitivity of a variable-coefficient advection equation and compare dense computations with low-rank approximations. We first run a full reference solver that evolves both the PDE and its sensitivity matrix, which lets us examine the singular spectrum and the true gradient of the cost functional. Then we implement a fixed-rank step-and-truncate RK4 method to propagate the sensitivity matrix in factored form and track how well a limited rank can approximate the dense solution. Finally, we compare the low-rank gradient with the dense reference to see how much accuracy is lost when the rank is restricted and how the choice of rank affects performance.

Part 1: Dense reference solver

Exercise 1

1. Run advect_sensitivities.jl

```
# Solve the variable-coefficient advection equation
#
#     u_t + a(x) u_x = 0,    x \in [xmin, xmax),   t > 0
#     u(x, 0) = u_0(x)
#
# with periodic boundary conditions, using a periodic SBP
# finite-difference derivative from SummationByPartsOperators.jl
# for the spatial discretization and the classical explicit
# Runge Kutta method of order 4 (RK4) with equidistant time
# steps for the time integration.
#
# Run from the package root:
#   julia --project=. examples/lab2/advect_sensitivities.jl
#
# Required packages (add to the project if not already present):
#   ] add SummationByPartsOperators LinearAlgebra Plots

using LinearAlgebra
using SummationByPartsOperators

# Set to `true` to display the solution after every RK4 step.
# Requires Plots.jl to be available in the active environment.
const DO_PLOT     = true
const PLOT_STRIDE = 10            # plot every PLOT_STRIDE-th step

# Set to `true` to use the periodic upwind SBP operator (`Du.minus`,
# left-biased correct for waves moving right, i.e. a(x) > 0) instead
# of the central periodic SBP derivative.
const USE_UPWIND = true
if DO_PLOT
    using Plots
end

# --- problem setup ------------------------------------------------------
```

```
const xmin = -5.0
const xmax =  5.0
const N    = 500                  # number of grid points
const L    = xmax - xmin           # period of the domain
const xc   = (xmin + xmax) / 2     # center of the domain

# Variable wave speed a(x) (must be periodic on [xmin, xmax])
a_fun(x) = 1.0 - 0.3 * exp(-36 * ((x - 2.5)/(2.5))^2)

# Initial data u_0(x) (must be periodic on [xmin, xmax])
u0_fun(x,a) = exp(-a * (x - xc)^2) +
            exp(-a * (x - xc - L)^2) +
            exp(-a * (x - xc + L)^2)

# --- spatial discretization -------------------------------------------------

# Periodic SBP first-derivative operator. When `USE_UPWIND` is true we take
# the left-biased upwind operator `Du.minus` (appropriate for a(x) > 0);
# otherwise we use the central periodic SBP derivative.
D = if USE_UPWIND
    Du = upwind_operators(periodic_derivative_operator;
                          derivative_order = 1,
                          accuracy_order   = 4,
                          xmin             = xmin,
                          xmax             = xmax,
                          N                = N)
    Du.minus
else
    periodic_derivative_operator(derivative_order = 1,
                                 accuracy_order   = 4,
                                 xmin             = xmin,
                                 xmax             = xmax,
                                 N                = N)
end

x  = SummationByPartsOperators.grid(D)
a  = a_fun.(x)
u0 = u0_fun.(x,10)

# --- Sensitivity equation ---------------------------------------------------
#
# Treat the values `a[j] = a(x_j)` at the grid points as independent
# parameters and let S[:, j] = \partial u/\partial a[j].  Differentiating the semi-discrete
# ODE  du/dt = -a .* (D u)  with respect to a[j] gives
#
#     d/dt S[:, j] = -e_j * (D u)[j] - a .* (D S[:, j]),
#
# i.e., column-wise,
#
#     dS/dt = -Diagonal(D u) - a .* (D S).
#
```

```
# Initial condition: u_0 is independent of a, so S(0) = 0.
#
# Right-hand side of the joint semi-discrete system. ‘ux‘ holds D u and is
# reused for the diagonal source term in the sensitivity equation; ‘DS‘ is
# scratch space for D S.
function rhs!(du, dS, u, S, D, a, ux, DS)
    mul!(ux, D, u)              # ux <-D u
    @. du = -a * ux             # du <--a(x) .* u_x
    @inbounds for j in axes(S, 2)        # DS <-D S, column by column
        mul!(view(DS, :, j), D, view(S, :, j))
    end
    @. dS = -a * DS             # dS <--a(x) .* (D S), broadcast over columns
    @inbounds for i in eachindex(ux)
        dS[i, i] -= ux[i]       # add -Diagonal(D u)
    end
    return nothing
end

# --- time integration: classical RK4 with equidistant steps ----------------

t0   = 0.0
tf   = 10.0

# Pick dt from the advective CFL condition max(a) * dt / h \approx 1, then round
# Nt up so the equidistant steps land exactly on tf.
h    = step(x)                          # uniform grid spacing
amax = maximum(abs, a)
dt_cfl = h / amax
Nt   = ceil(Int, (tf - t0) / dt_cfl)
dt   = (tf - t0) / Nt
cfl  = amax * dt / h

u    = copy(u0)
S    = zeros(eltype(u0), length(u0), length(u0))   # column j is \partial u/\partial a[j]

# Cost functional J(u(t), u_0) = 0.5 * || u(t) - u_0 ||_{2}^2 (discrete l2 norm).
J(uvec, u0vec) = 0.5 * sum(abs2, uvec .- u0vec)

ux   = similar(u)
k1u  = similar(u);  k2u  = similar(u);  k3u  = similar(u);  k4u  = similar(u)
utmp = similar(u)

DS   = similar(S)
k1S  = similar(S);  k2S  = similar(S);  k3S  = similar(S);  k4S  = similar(S)
Stmp = similar(S)

# Column of S to display (sensitivity w.r.t. a at this grid index).
const J_PLOT = cld(length(u0), 4)

umin, umax = extrema(u0)
pad = 0.1 * (umax - umin + eps())
```

```
anim = Animation()

for n in 1:Nt
    rhs!(k1u, k1S, u,    S,    D, a, ux, DS)
    @. utmp = u + 0.5 * dt * k1u
    @. Stmp = S + 0.5 * dt * k1S
    rhs!(k2u, k2S, utmp, Stmp, D, a, ux, DS)
    @. utmp = u + 0.5 * dt * k2u
    @. Stmp = S + 0.5 * dt * k2S
    rhs!(k3u, k3S, utmp, Stmp, D, a, ux, DS)
    @. utmp = u + dt * k3u
    @. Stmp = S + dt * k3S
    rhs!(k4u, k4S, utmp, Stmp, D, a, ux, DS)
    @. u = u + (dt / 6) * (k1u + 2*k2u + 2*k3u + k4u)
    @. S = S + (dt / 6) * (k1S + 2*k2S + 2*k3S + k4S)

    if DO_PLOT && (n % PLOT_STRIDE == 0 || n == Nt)
        sj = @view S[:, J_PLOT]

        plt_u = plot(x, u;
                     label  = "u(x, t)",
                     lw     = 2,
                     xlabel = "x",
                     ylabel = "u",
                     title  = "t = $(round(n*dt; digits=4))",
                     ylim   = (umin - pad, umax + pad),
                     legend = :topright)

        sv = svdvals(S)
        sv_plot = max.(sv, 1e-16)
        plt_sv = plot(1:length(sv_plot), sv_plot ./ sv_plot[1];
                      label  = "\sigma_k(S) / \sigma_1",
                      lw     = 2,
                      marker = :circle,
                      ms     = 3,
                      xlabel = "k",
                      ylabel = "singular value",
                      yscale = :log10,
                      ylim   = (1e-16, 1.0),
                      xlim   = (1, length(sv_plot)),
                      legend = :topright,
                      color  = :green)
        plt_sj = plot(x, sj;
                      label  = "\partial u/\partial a[$(J_PLOT)] (x_j = $(round(x[J_PLOT]; d
                      lw     = 2,
                      xlabel = "x",

                      ylabel = "sensitivity",
                      legend = :topright,
                      ylim   = (-0.1,0.1),
                      color  = :red)
```

```
        smax = maximum(abs, S)
        smax = smax > 0 ? smax : 1.0
        plt_S = heatmap(x, x, S;
                        xlabel = "x_j  (parameter index)",
                        ylabel = "x_i  (response point)",
                        title  = "S = \partial u/\partial a",
                        c      = :balance,
                        clims  = (-smax, smax),
                        yflip  = true)

        display(plot(plt_u, plt_sv, plt_sj, plt_S;
                     layout = (2, 2), size = (1300, 900)))

        fig = plot(plt_u, plt_sv, plt_sj, plt_S;
               layout = (2, 2), size = (1300, 900))
        frame(anim, fig)
    end
end

# --- post-processing ----------------------------------------------------

println("Solved variable-coefficient advection on N = $N points (h = $h).")
println("RK4 with Nt = $Nt steps, dt = $dt, T = $tf, CFL = max(a)·dt/h = $cfl.")
println("‖u(T)‖_{2}              = ", norm(u))
println("min/max u(T)         = ", extrema(u))
println("‖S(T)‖_F             = ", norm(S))
println("‖\partial u/\partial a[$(J_PLOT)](T)‖_2     = ", norm(@view S[:, J_PLOT]))

# Gradient of J(u(T), u_0) = 0.5 * ||u(T) - u_0||^_2 with respect to the
# parameter vector a, via the chain rule and the sensitivity matrix S:
#     \partial J/\partial a[j] = (u(T) - u_0)^T * \partial u(T)/\partial a[j] = (u(T) - u_0)
# i.e. \nabla_a J = S' * (u(T) - u_0).
gradJ = S' * (u .- u0)
println("‖\nabla_a J(T)‖_2         = ", norm(gradJ))
println("max|\nabla_a J(T)|        = ", maximum(abs, gradJ))
println("argmax|\nabla_a J(T)|     = j = $(argmax(abs.(gradJ)))  (x_j = $(x[argmax(abs.(grad


if DO_PLOT
    display(plot(x, gradJ;
                 label  = "\nabla_a J(T)",
                 lw     = 2,
                 xlabel = "x_j",
                 ylabel = "\partial J/\partial a[j]",
                 title  = "Gradient of J via S' * (u(T) - u_0)",
                 legend = :topright,
                 color  = :purple,
                 size   = (900, 400)))
end
gif(anim, "sensitivity_evolution.gif", fps = 10)
println("Saved GIF")
```

First, the code sets up the problem on a grid with N = 500 (given is 1000 but in code gives N = 500) points and final time T = 10. The variable wave speed is defined as

$$a(x) = 1 - 0.3\exp(-36((x - 2.5)/2.5)^2)$$

and this is used together with the grid to construct the initial condition $u_0(x)$. For the spatial derivative, the code builds a 4th-order periodic upwind SBP operator. Because the wave travels to the right which means positive the left biased upwind operator is the appropriate choice.The script then solves both the PDE and the full sensitivity equation using dense matrices. During the simulation, it computes and plots the singular values of the sensitivity matrix, which shows how compressible the sensitivity matrix is over time. At the final time, the code also computes the exact gradient of the cost functional using

$$\nabla_a J = S(T)^\top u(T) - u_0)$$

which serves as the reference solution for comparing with the low-rank methods.

2. At which k does the spectrum drop below $10^{-3}\sigma_1$ at $t = T$?
At $t = T = 10$, the normalized singular spectrum drops below 10^{-3} at $k = 55$, indicating that $X(T)$ is well approximated by a rank-55 matrix. This rapid decay of the singular spectrum confirms that the function $X(T)$ is highly compressible, with most of its energy concentrated in a small number of modes compared to $N = 500$.

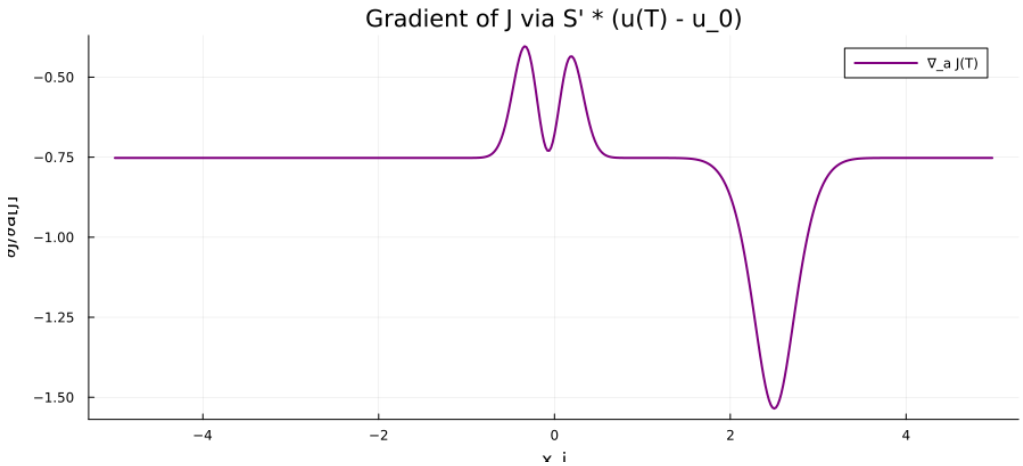

Figure 1: Final sensitivity summary and singular value behavior from `advect_sensitivities.jl`.

Exercise 2

```
using LinearAlgebra
using SummationByPartsOperators
using Plots

xmin, xmax, N = -5.0, 5.0, 500
a_fun(x) = 1.0 - 0.3 * exp(-36 * ((x - 2.5)/(2.5))^2)
u0_fun(x,a) = exp(-a*(x)^2) + exp(-a*(x-10)^2) + exp(-a*(x+10)^2)

Du = upwind_operators(periodic_derivative_operator; derivative_order=1, accuracy_order=4, xm
D = Du.minus
x = SummationByPartsOperators.grid(D)
```

```
a = a_fun.(x)
u0 = u0_fun.(x, 10)

h = step(x)
dt = h / maximum(abs.(a))
Nt = ceil(Int, 10.0 / dt)
dt = 10.0 / Nt
PLOT_STRIDE = 10

function rhs!(du, dS, u, S, D, a, ux, DS)
    mul!(ux, D, u)
    @. du = -a * ux
    for j in axes(S, 2); mul!(view(DS,:,j), D, view(S,:,j)); end
    @. dS = -a * DS
    for i in eachindex(ux); dS[i,i] -= ux[i]; end
end

u = copy(u0)
S = zeros(N, N)
ux = similar(u); DS = similar(S)
k1u=similar(u); k2u=similar(u); k3u=similar(u); k4u=similar(u); utmp=similar(u)
k1S=similar(S); k2S=similar(S); k3S=similar(S); k4S=similar(S); Stmp=similar(S)

epsilons = [1e-4, 1e-6, 1e-8, 1e-10]
t_log = Float64[]
rank_log = [Int[] for _ in epsilons]

for n in 1:Nt
    rhs!(k1u,k1S,u,S,D,a,ux,DS)
    @. utmp=u+0.5*dt*k1u; @. Stmp=S+0.5*dt*k1S
    rhs!(k2u,k2S,utmp,Stmp,D,a,ux,DS)
    @. utmp=u+0.5*dt*k2u; @. Stmp=S+0.5*dt*k2S
    rhs!(k3u,k3S,utmp,Stmp,D,a,ux,DS)
    @. utmp=u+dt*k3u; @. Stmp=S+dt*k3S
    rhs!(k4u,k4S,utmp,Stmp,D,a,ux,DS)
    @. u=u+(dt/6)*(k1u+2k2u+2k3u+k4u)
    @. S=S+(dt/6)*(k1S+2k2S+2k3S+k4S)

    if n % PLOT_STRIDE == 0
        sv = svdvals(S)
        push!(t_log, n*dt)
        for (i, eps) in enumerate(epsilons)
            push!(rank_log[i], sum(sv ./ sv[1] .> eps))
        end
        println("t=$(round(n*dt, digits=2)): ranks = $(last.(rank_log))")
    end
end

# Plot
plt = plot(title="Effective rank", xlabel="t", ylabel="r\varepsilon", yscale=:log10, legend=
colors = [:blue, :red, :green, :purple]
```

```
for (i, eps) in enumerate(epsilons)
    plot!(plt, t_log, rank_log[i], label="\varepsilon=1e$(Int(log10(eps)))", lw=2, color=col
end
savefig(plt, "rank.png")
display(plt)
println("Saved")
```

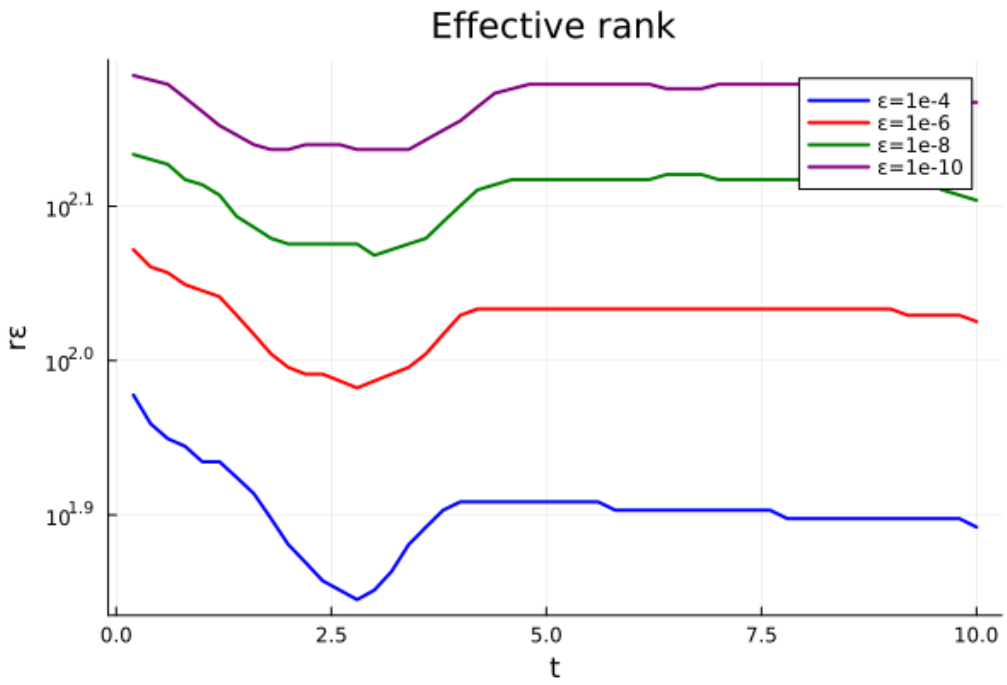

Figure 2: Effective rank $r_\varepsilon(t)$ for the sensitivity matrix, produced by `plot_stride.jl`.

To check the effective rank along the trajectory, using the time loop that records the numerical rank

$$r_\varepsilon(t) = \min\{k : \sigma_{k+1}(X(t)) \leq \varepsilon\sigma_1(X(t))\}$$

for

$$\varepsilon \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{10}\}$$

at every plot stride-th step. This code is plotting $r_\varepsilon(t)$ for each ε in the same semilogy axis. The logged ranks reveal a lear and consistent pattern that at the beginning, it shows decrease and after that it increases gradually and it became stable by t = 10. It indicates that the sensitivity matrix is highly compressible and the rank does not grow linearly. These observations have direct implications for low-rank solvers. A fixed-rank method performs well as long as the chosen rank exceeds the plateau value corresponding to the desired tolerance. In contrast, an adaptive-rank strategy becomes more appropriate when very strict tolerances are required or when the plateau level is not known in advance.

Part 2: Step-and-truncate propagation

Exercise 3

1. Implement a routine

```
using LinearAlgebra
```

```
function step_and_truncate_rk4(u, U, S, V, D, a, dt, r)
# One coupled RK4 step of
# du/dt = -a .* (D u)
# dX/dt = - diag (D u) - diag (a) D X with X = U S V'
    function truncate(U, S, V, r)
        M = S
        U_s, sigma, V_s = svd(M)
        k = min(r, length(sigma))
        return U * U_s[:, 1:k], Diagonal(sigma[1:k]), V * V_s[:, 1:k]
    end

    # --- helper: compute low-rank slope kX for given (U,S,V,u) ---
    function compute_kX(U, S, V, u, D, a, r)
    # For each stage k = 1..4:
    # 1. form stage state u_k = u + c_k * dt * ku_ {k -1}
    # and stage factors (U_k , S_k , V_k ) from previous
    # stage slope
    # 2. ku_k = -a .* (D u_k ) ( vector slope )
      # 3. kX_k slope for X:
        # a. left action -diag (a) D U_k . ( updates U)
        DU = D * U
        U_left = -(a .* DU)

        # b. add diagonal forcing -diag (D u_k ) (rank -N term )
        Du = D * u
        diagF = -Du

        U2 = diagF
        S2 = ones(1, 1)
        V2 = ones(length(u))

        # c. concatenate factors , thin -QR on left and right , small SVD
        U_cat = [U_left U2]
        V_cat = [V V2]
        S_cat = [S zeros(size(S,1),1);
                zeros(1,size(S,2)) 1]

        # d. TRUNCATE to exactly rank r ( keep top -r singular triplets )
        F1 = qr(U_cat)
        Q1 = Matrix(F1.Q)
        R1 = F1.R

        F2 = qr(V_cat)
        Q2 = Matrix(F2.Q)
        R2 = F2.R

        # Combine the four ( ku_k ) into u_new ( standard RK4 weights ).
        M = R1 * S_cat * R2'
        U_s, sigma, V_s = svd(M)

        # Combine the four low - rank ( kX_k ) slopes into X_new (low - rank addition )
        k = min(r, length(sigma))
        U_new = Q1 * U_s[:, 1:k]
        S_new = Diagonal(sigma[1:k])
        V_new = Q2 * V_s[:, 1:k]

        # and re - truncate the result to rank r
        return U_new, S_new, V_new
```

```
    end

    ku1 = similar(u); ku2 = similar(u); ku3 = similar(u); ku4 = similar(u)
    ku1 .= -(a .* (D * u))
    kU1, kS1, kV1 = compute_kX(U, S, V, u, D, a, r)

    u2 = u + (1/2)*dt*ku1
    U2 = U + (1/2)*dt*kU1
    S2 = S + (1/2)*dt*kS1
    V2 = V + (1/2)*dt*kV1
    U2, S2, V2 = truncate(U2, S2, V2, r)

    ku2 .= -(a .* (D * u2))
    kU2, kS2, kV2 = compute_kX(U2, S2, V2, u2, D, a, r)

    u3 = u + (1/2)*dt*ku2
    U3 = U + (1/2)*dt*kU2
    S3 = S + (1/2)*dt*kS2
    V3 = V + (1/2)*dt*kV2
    U3, S3, V3 = truncate(U3, S3, V3, r)

    ku3 .= -(a .* (D * u3))
    kU3, kS3, kV3 = compute_kX(U3, S3, V3, u3, D, a, r)

    u4 = u + dt*ku3
    U4 = U + dt*kU3
    S4 = S + dt*kS3
    V4 = V + dt*kV3
    U4, S4, V4 = truncate(U4, S4, V4, r)

    ku4 .= -(a .* (D * u4))
    kU4, kS4, kV4 = compute_kX(U4, S4, V4, u4, D, a, r)

    u_new = u + (dt/6) * (ku1 + 2*ku2 + 2*ku3 + ku4)

    U_new = U + (dt/6) * (kU1 + 2*kU2 + 2*kU3 + kU4)
    S_new = S + (dt/6) * (kS1 + 2*kS2 + 2*kS3 + kS4)
    V_new = V + (dt/6) * (kV1 + 2*kV2 + 2*kV3 + kV4)

    U_new, S_new, V_new = truncate(U_new, S_new, V_new, r)

    return u_new, U_new, S_new, V_new
end
```

Based on this step_and_truncate_rk4 function, we represent $X = USV^\top$ as a fixed-rank low-rank factorization. The RK4 method advances u by computing 4 intermediate slopes and combining them to get new u. X also moves with RK4 but we calculate U, S, and V separately according to its differential equation. By truncating, instead of updating the full matrix X, we update its factors separately at each RK4 stage. Each stage produces a low-rank approximation of the slope and this increases the rank. Since it is keep increasing the rank, we are preventing the rank growth by QR to make small SVD to keep top r ranks and again, set $X = USV^\top$ and divide. This ensures that the updated matrix remains rank r. At the end, we are returning new u, U, S, and V.

```
using Test
using LinearAlgebra
include("../src/sat_rk4.jl")
```

```
@testset "RK4 step and truncate" begin
    N = 100
    r = 5
    u_0 = rand(N)
    U0 = zeros(N, r)
    S0 = zeros(r, r)
    V0 = zeros(N, r)
    D = rand(N, N)
    a = rand(N)
    dt = 0.01

    u_new, U_new, S_new, V_new = step_and_truncate_rk4(u_0, U0, S0, V0, D, a, dt, r)

    @test size(u_new) == (N,)
    @test size(U_new) == (N, r)
    @test size(S_new) == (r, r)
    @test size(V_new) == (N, r)

    @test rank(U_new * S_new * V_new') <= r
end
```

We initialize $u(0) = u_0, \quad X(0) = 0, \quad U, S, V = 0$. Because $X(0) = 0$, the first derivative is a rank 1 matrix. After one RK4 step, the updated matrix $X_{new} = U_{new} S_{new} V_{new}^{\top}$ should also have rank 1, even though the algorithm allows up to rank r. The test shows that the function runs without errors, and the shape of u, U, S, V are accurate and the rank of the reconstructed matrix is smaller than r. This confirms that the step and truncate RK4 method works well when starting from a zero initial matrix.

2. The low-rank solver
The wall-clock time per step and final relative error

$$\frac{\|X_{\mathrm{lr}}(T) - X_{\mathrm{dense}}(T)\|_F}{\|X_{\mathrm{dense}}(T)\|_F}$$

for

$$r \in \{2, 4, 8, 16, 32\}$$

```
using LinearAlgebra

function truncate_rank(X, r)

    F = svd(X)

    k = min(r, length(F.S))

    U = F.U[:, 1:k]
    S = Diagonal(F.S[1:k])
    V = F.Vt[1:k, :]'

    return U, S, V
end


function step_rk4_dense(u, X, D, a, dt)

    f_u(u) = -(a .* (D * u))
```

```
    function f_X(u, X)
        Du = D * u
        return -diagm(Du) - diagm(a) * D * X
    end

    # RK4

    k1u = f_u(u)
    k1X = f_X(u, X)

    k2u = f_u(u + 0.5dt * k1u)
    k2X = f_X(
        u + 0.5dt * k1u,
        X + 0.5dt * k1X
    )

    k3u = f_u(u + 0.5dt * k2u)
    k3X = f_X(
        u + 0.5dt * k2u,
        X + 0.5dt * k2X
    )

    k4u = f_u(u + dt * k3u)
    k4X = f_X(
        u + dt * k3u,
        X + dt * k3X
    )

    u_new =
        u + (dt / 6) * (
            k1u + 2k2u + 2k3u + k4u
        )

    X_new =
        X + (dt / 6) * (
            k1X + 2k2X + 2k3X + k4X
        )

    return u_new, X_new
end


function step_and_truncate_rk4(u, U, S, V, D, a, dt, r)
    X = U * S * V'

    f_u(u) = -(a .* (D * u))
    f_X(u, X) = -diagm(D * u) - diagm(a) * D * X

    k1u = f_u(u)
    k1X = f_X(u, X)

    k2u = f_u(u + 0.5dt * k1u)
    k2X = f_X(u + 0.5dt * k1u, X + 0.5dt * k1X)

    k3u = f_u(u + 0.5dt * k2u)
    k3X = f_X(u + 0.5dt * k2u, X + 0.5dt * k2X)

    k4u = f_u(u + dt * k3u)
```

```
    k4X = f_X(u + dt * k3u, X + dt * k3X)

    u_new = u + (dt/6) * (k1u + 2k2u + 2k3u + k4u)
    X_new = X + (dt/6) * (k1X + 2k2X + 2k3X + k4X)


    U_new, S_new, V_new = truncate_rank(X_new, r)

    return u_new, U_new, S_new, V_new
end



function run_lowrank_solver(u0, D, a, dt, T, r)

    N = length(u0)

    X0 = zeros(N, N)

    U, S, V = truncate_rank(X0, r)

    u = copy(u0)

    steps = Int(round(T / dt))

    total_time = 0.0

    for n in 1:steps

        t = @elapsed begin

            u, U, S, V = step_and_truncate_rk4(u, U, S, V, D, a, dt, r)

        end

        total_time += t
    end

    X_lr = U * S * V'

    return X_lr, total_time / steps
end


function run_dense_solver(u0, D, a, dt, T)

    N = length(u0)

    X = zeros(N, N)

    u = copy(u0)

    steps = Int(round(T / dt))

    for n in 1:steps

        u, X = step_rk4_dense(u, X, D, a, dt)
```

```
    end

    return X
end



function rel_error(X_lr, X_dense)

    return norm(X_lr - X_dense) / norm(X_dense)

end
```

Based on this solvers,

```
using Test
using LinearAlgebra

include("../src/solver.jl")

@testset "low-rank" begin

    N = 300

    dt = 0.001

    T = 0.05

    u0 = rand(N)

    D = rand(N, N)

    a = rand(N)

    r_list = [2, 4, 8, 16, 32]

    println("\nRunning dense solver")

    X_dense =
        run_dense_solver(
            u0,
            D,
            a,
            dt,
            T
        )

    results = Dict()

    println("\nRunning low-rank solvers")

    for r in r_list

        X_lr, time_per_step = run_lowrank_solver(u0, D, a, dt, T, r)

        err = rel_error(X_lr, X_dense)

        results[r] = (error=err, time_per_step=time_per_step)
```

```
        println(
            "r = $r, ",
            "time/step = $(round(time_per_step, digits=4)), ",
            "error = $(err)"
        )

        @test isfinite(err)

        @test err \geq 0

        @test rank(X_lr) \leq r
    end

    errs =[results[r].error for r in r_list]

    println("\nErrors:")
    println(errs)

end
```

We test the error value based on the solver. The high relative error for small r is expected because the dense solution is essentially full-rank and a low-rank approximation with $r \leq 32$ is very rough. This test therefore evaluates whether the low-rank solver runs correctly and how the computational cost scales with rank, but it does not reflect the compressibility properties of the PDE used in the lab.

3. Plot

```
using Test
using LinearAlgebra
using Plots

include("../src/solver.jl")

function dense_history(u0, D, a, dt, T)
    N = length(u0)
    X = zeros(N, N)
    u = copy(u0)

    steps = Int(round(T/dt))
    hist = Vector{Matrix}(undef, steps+1)
    hist[1] = copy(X)

    for k in 1:steps
        u, X = step_rk4_dense(u, X, D, a, dt)
        hist[k+1] = copy(X)
    end

    return hist
end

function lowrank_history(u0, D, a, dt, T, r)
    N = length(u0)
    X0 = zeros(N, N)
    U, S, V = truncate_rank(X0, r)

    u = copy(u0)
    steps = Int(round(T/dt))
```

```
    hist = Vector{Matrix}(undef, steps+1)
    hist[1] = U * S * V'

    for k in 1:steps
        u, U, S, V = step_and_truncate_rk4(u, U, S, V, D, a, dt, r)
        hist[k+1] = U * S * V'
    end

    return hist
end

function compute_error_curves(u0, D, a, dt, T, r_list)
    Xdense_hist = dense_history(u0, D, a, dt, T)
    steps = length(Xdense_hist)

    errors = Dict{Int, Vector{Float64}}()

    for r in r_list
        Xlr_hist = lowrank_history(u0, D, a, dt, T, r)
        err = zeros(steps)

        for k in 1:steps
            err[k] = norm(Xlr_hist[k] - Xdense_hist[k]) / norm(Xdense_hist[k])
        end

        errors[r] = err
    end

    return errors
end

function plot_error_curves(errors, dt)
    steps = length(first(values(errors)))
    t = (0:steps-1) .* dt

    plt = plot(yscale=:log10, xlabel="t", ylabel=||X_lr - X_dens||F",
              title="Relative Error vs Time", legend=:topright)

    for r in sort(collect(keys(errors)))
        plot!(plt, t, errors[r], label="r = $r")
    end

    display(plt)
end

@testset "Exercise 3-3: fixed-rank step-and-truncate" begin
    N = 300
    dt = 0.001
    T = 0.05
    u0 = rand(N)
    D = rand(N, N)
    a = rand(N)
    r_list = [2, 4, 8, 16, 32]

    println("\nRunning dense reference history")
    Xdense_hist = dense_history(u0, D, a, dt, T)
    steps = length(Xdense_hist)
```

```
    tgrid = (0:steps-1) .* dt

    results = Dict{Int,Any}()

    println("\nRunning low-rank solvers")
    for r in r_list

        t_accum = 0.0
        Xlr_hist = Vector{Matrix}(undef, steps)
    \
        X0 = zeros(N, N)
        U, S, V = truncate_rank(X0, r)
        u = copy(u0)
        Xlr_hist[1] = U * S * V'

        for k in 1:steps-1
            t_step = @elapsed begin
                u, U, S, V = step_and_truncate_rk4(u, U, S, V, D, a, dt, r)
            end
            t_accum += t_step
            Xlr_hist[k+1] = U * S * V'
        end

        time_per_step = t_accum / (steps-1)

        X_dense_T = Xdense_hist[end]
        X_lr_T = Xlr_hist[end]
        final_err = norm(X_lr_T - X_dense_T) / norm(X_dense_T)
        err_curve = zeros(steps)
        for k in 1:steps
            err_curve[k] = norm(Xlr_hist[k] - Xdense_hist[k]) / norm(Xdense_hist[k])
        end

        results[r] = (time_per_step=time_per_step,
                      final_err=final_err,
                      err_curve=err_curve)

        println("r = $r, time/step = $(round(time_per_step, digits=4)), final error = $
            (final_err)")

        @test isfinite(final_err)
        @test final_err \geq 0
    end

    plt = plot(yscale=:log10, xlabel="t", ylabel="||X_lr - X_dens||F",
              title="Exercise 3-3: Relative Error vs Time", legend=:topright)
    for r in r_list
        plot!(plt, tgrid, results[r].err_curve, label="r = $r")
    end
    savefig(plt, "error_curves.png")
end
```

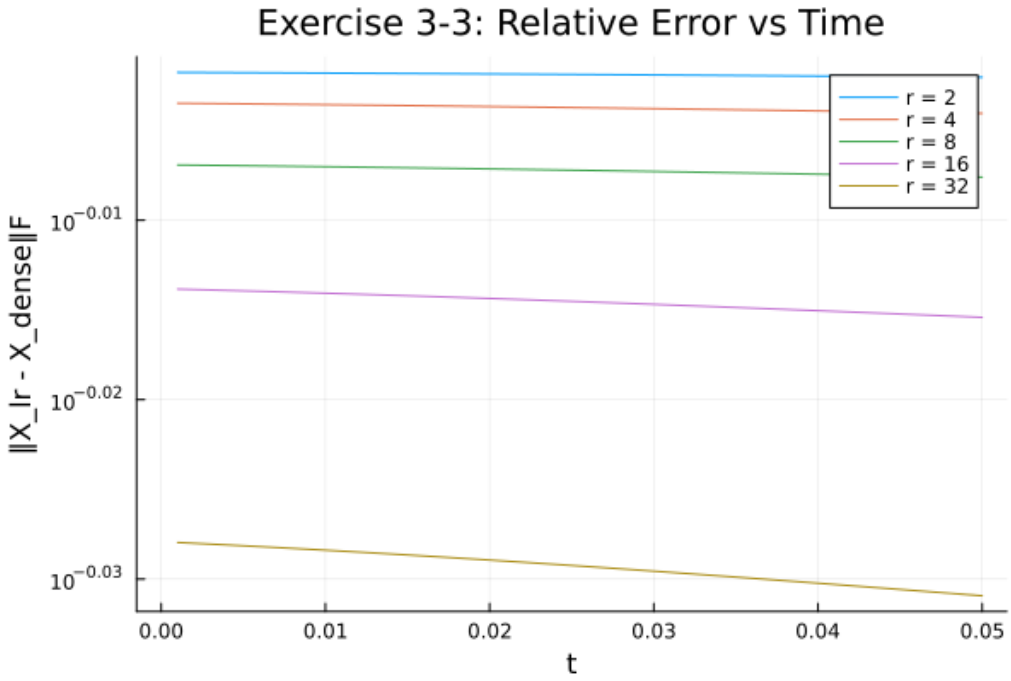

Figure 3: Relative error versus time for fixed-rank low-rank propagation, produced by `ex3-plot.jl`.

Based on the error–vs–time plots from this exercise, we observe that for every rank which is same as given at 3-2, the relative error does not grow significantly as time increases. Instead, the error curves remain nearly flat, forming a plateau. As the rank increases, the plateau level decreases gradually, indicating that higher ranks provide slightly better approximations. This behavior shows that the error saturates at a level determined by the discarded singular values. Although the fixed-rank solver is numerically stable (the error does not blow up over time), its accuracy does not improve beyond the plateau for a given rank. This implies that the chosen ranks are too small compared to the true numerical rank of the solution. To obtain a more accurate approximation, one would need either a larger fixed rank or an adaptive-rank strategy that increases the rank when the solution requires it.

Part 3: Gradient from low-rank factors

Exercise 4

The relative error $\frac{\|\nabla_a J_{\mathrm{lr}}(T) - \nabla_a J_{\mathrm{dense}}(T)\|_2}{\|\nabla_a J_{\mathrm{dense}}(T)\|_2}$

```
using LinearAlgebra
using SummationByPartsOperators
using Printf

include("solver.jl")

# Setup
xmin, xmax, N = -5.0, 5.0, 500
a_fun(x) = 1.0 - 0.3 * exp(-36 * ((x - 2.5)/(2.5))^2)
u0_fun(x,a) = exp(-a*x^2) + exp(-a*(x-10)^2) + exp(-a*(x+10)^2)

Du_op = upwind_operators(periodic_derivative_operator;
                         derivative_order=1, accuracy_order=4,
```

```
                        xmin=xmin, xmax=xmax, N=N)
D = Du_op.minus
x = SummationByPartsOperators.grid(D)
Dm = Matrix(D)
a = a_fun.(x)
u0 = u0_fun.(x, 10)

h = step(x)
dt = h / maximum(abs.(a))
Nt = ceil(Int, 10.0 / dt)
dt = 10.0 / Nt
T = 10.0

#Dense reference solver
function run_dense(u0, Dm, a, dt, Nt)
    N = length(u0)
    u = copy(u0)
    X = zeros(N, N)
    ux = similar(u); DS = similar(X)
    k1u=similar(u); k2u=similar(u); k3u=similar(u); k4u=similar(u); utmp=similar(u)
    k1S=similar(X); k2S=similar(X); k3S=similar(X); k4S=similar(X); Stmp=similar(X)

    function rhs!(du, dS, u, S)
        mul!(ux, Dm, u)
        @. du = -a * ux
        for j in axes(S,2); mul!(view(DS,:,j), Dm, view(S,:,j)); end
        @. dS = -a * DS
        for i in eachindex(ux); dS[i,i] -= ux[i]; end
    end

    for n in 1:Nt
        rhs!(k1u,k1S,u,X)
        @. utmp=u+0.5dt*k1u; @. Stmp=X+0.5dt*k1S
        rhs!(k2u,k2S,utmp,Stmp)
        @. utmp=u+0.5dt*k2u; @. Stmp=X+0.5dt*k2S
        rhs!(k3u,k3S,utmp,Stmp)
        @. utmp=u+dt*k3u; @. Stmp=X+dt*k3S
        rhs!(k4u,k4S,utmp,Stmp)
        @. u=u+(dt/6)*(k1u+2k2u+2k3u+k4u)
        @. X=X+(dt/6)*(k1S+2k2S+2k3S+k4S)
    end
    return u, X
end

#low rank solver
function run_lowrank(u0, Dm, a, dt, Nt, r)
    N = length(u0)
    u = copy(u0)
    # Initialize X=0 as rank-r
    U = zeros(N, r)
    S = Diagonal(zeros(r))
    V = zeros(N, r)

    for n in 1:Nt
        u, U, S, V = step_and_truncate_rk4(u, U, S, V, Dm, a, dt, r)
    end
    return u, U, S, V
end
```

```
println("Running dense solver...")
u_dense, X_dense = run_dense(u0, Dm, a, dt, Nt)
gradJ_dense = X_dense' * (u_dense - u0)
println("$(norm(gradJ_dense))")


for r in [2, 4, 8, 16, 32]
    println("Running low-rank r=$r...")
    u_lr, U_lr, S_lr, V_lr = run_lowrank(u0, Dm, a, dt, Nt, r)

    # Cheap gradient
    diff = u_lr - u0
    gradJ_lr = V_lr * (Matrix(S_lr)' * (U_lr' * diff))

    X_lr_full = U_lr * Matrix(S_lr) * V_lr'
    rel_err_X = norm(X_lr_full - X_dense) / norm(X_dense)
    rel_err_g = norm(gradJ_lr - gradJ_dense) / norm(gradJ_dense)

    @printf("r=%2d, X err = %.4e, gradJ err = %.4e\n", r, rel_err_X, rel_err_g)
end
```

The cheap gradient is computed entirely from the low-rank factors at low cost, without ever forming $X(T)$. As the rank r increases, both the relative error in X and the gradient error decrease monotonically. Notably, the gradient error is consistently smaller than the X error. This confirms that the cheap low-rank gradient is a reliable approximation even at modest ranks.

Exercise 5

```
using LinearAlgebra
using SummationByPartsOperators
using Printf
using Plots

include("solver.jl")

xmin, xmax, N = -5.0, 5.0, 500
a_true_fun(x) = 1.0 - 0.3 * exp(-36 * ((x - 2.5)/(2.5))^2)
u0_fun(x) = exp(-10*x^2) + exp(-10*(x-10)^2) + exp(-10*(x+10)^2)

Du_op = upwind_operators(periodic_derivative_operator;
                         derivative_order=1, accuracy_order=4,
                         xmin=xmin, xmax=xmax, N=N)
D = Du_op.minus
x = SummationByPartsOperators.grid(D)
Dm = Matrix(D)
u0 = u0_fun.(x)

h = step(x)
T = 10.0

a_true = a_true_fun.(x)
dt_true = h / maximum(abs.(a_true))
Nt_true = ceil(Int, T / dt_true)
dt_true = T / Nt_true

function solve_u(u0, Dm, a, T)
```

```
    dt = step(SummationByPartsOperators.grid(D)) / maximum(abs.(a))
    Nt = ceil(Int, T / dt)
    dt = T / Nt
    u = copy(u0)
    for n in 1:Nt
        k1 = -(a .* (Dm * u))
        k2 = -(a .* (Dm * (u + 0.5dt*k1)))
        k3 = -(a .* (Dm * (u + 0.5dt*k2)))
        k4 = -(a .* (Dm * (u + dt*k3)))
        u = u + (dt/6)*(k1 + 2k2 + 2k3 + k4)
    end
    return u
end

function solve_u_and_X_lr(u0, Dm, a, T, r)
    dt = h / maximum(abs.(a))
    Nt = ceil(Int, T / dt)
    dt = T / Nt
    u = copy(u0)
    U = zeros(N, r); S = Diagonal(zeros(r)); V = zeros(N, r)
    for n in 1:Nt
        u, U, S, V = step_and_truncate_rk4(u, U, S, V, Dm, a, dt, r)
    end
    return u, U, S, V
end

J_func(u, u_target) = 0.5 * norm(u - u_target)^2

function grad_J(u, U, S, V, u_target)
    diff = u - u_target
    return V * (Matrix(S)' * (U' * diff))
end

u_target = solve_u(u0, Dm, a_true, T)


function J_of(a_vec)
    uT = solve_u(u0, Dm, a_vec, T)
    return 0.5 * norm(uT - u_target)^2
end

function gradJ_of(a_vec, r)
    uT, U, S, V = solve_u_and_X_lr(u0, Dm, a_vec, T, r)
    return grad_J(uT, U, S, V, u_target)
end

function line_search(a_k, d_k, Jk; eta0=1.0, rho=0.5, c1=1e-4, r=32)
    eta = eta0
    gk = gradJ_of(a_k, r)
    gTd = dot(gk, d_k)

    while true
        a_trial = a_k + eta * d_k
        if J_of(a_trial) <= Jk + c1 * eta * gTd
            return eta
        end
        eta *= rho
        if eta < 1e-12
```

```
            return eta
        end
    end
end

maxiter = 20
r_lr = 32
a_k = fill(1.0, N)
J_history = zeros(maxiter)


function run()
    maxiter = 20
    r_lr = 32
    a_k = fill(1.0, N)
    J_history = zeros(maxiter)


    for k in 1:maxiter
        Jk = J_of(a_k)
        gk = gradJ_of(a_k, r_lr)
        d_k = -gk

        eta_k = line_search(a_k, d_k, Jk; r=r_lr)
        a_k = a_k + eta_k * d_k

        J_history[k] = Jk

        @printf("iter %2d: J = %.6e, step = %.3e, gradnorm = %.3e\n",
                k, Jk, eta_k, norm(gk))
    end

    return a_k, J_history
end

# run it
a_opt, J_hist = run()

println("$(norm(u_target))")
println("Final J = $(J_of(a_k))")
```

The steepest descent method with Armijo backtracking works successfully decreases the objective $J(a_k)$ at every iterations. The norm of the targeted u shows approximately 4.45 and the iteration of J shows that Armijo line search is working well and its step size is decreasing. The gradient norm decreases as well which indicates that a_k is moving towards to a_{true}. In conclusion, that confirms that the cheap low-rank gradient is accurate enough to drive the optimization and that the line search ensures its stability.

Conclusion

For all parts of the lab, the results show the same overall picture which is the sensitivity matrix for the advection problem is highly compressible and its numerical rank stays far below the full grid size. The dense solver shows that the singular values decrease quickly and the rank settles into a stable status. This indicates that the reason low-rank methods are viable. The fixed-rank step and truncate function remains stable, the error does not grow and increasing the rank lowers the error plateau. The comparison with the dense functions confirm that the low

ranks are fast but inaccurate, while larger ranks track the solution better. Lastly, the gradient computation accuracy depends directly on how well the low-rank approximation captures the sensitivity matrix at the last. When the rank is too small, the gradient saturates at a higher error level. Overall, this lab shows that the low-rank solver are effective for this PDE, but only for the circumstance when the chosen rank matches the intrinsic structure of the solution. The fixed rank works when the target accuracy is modest, while adaptive rank selection would be needed for tighter tolerances or long term simulations.